

#AndroidHackingMonth: Introduction to Android Hacking by @Oxteknogeek

February 13th, 2020

Joel Margolis



Discover the 15% Ad
What Sets Top CISOs Apa

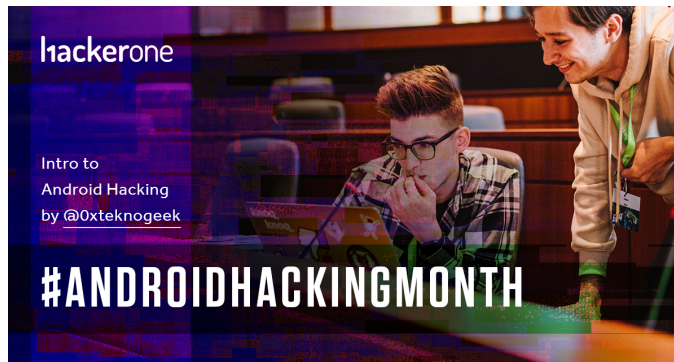
[Read the report →](#)

Subscribe

The latest from HackerOr

[Submit →](#)

We will handle your contact details in lin
[Privacy Policy](#). If you prefer not to receiv
emails from us, you can opt-out of all m
communications or customize your pref



When I first started mobile hacking, it felt a lot like the wild west. There were very few public resources, blog posts, tools, or communities, and everything was extremely hush-hush. Five years later, things have finally started to change....a little. However, I would still say that there is a major knowledge gap in the mobile security space that makes it easy for experts to excel and beginners to fail. As some people may know, I belong to a rare breed of hackers who focus primarily on mobile application security. I end up getting A LOT of questions about mobile hacking, ranging anywhere from "what tools do you recommend" to "bounty plz".

The main goal of this post is going to be to provide an introduction to mobile hacking (Android specifically). It will cover how I approach apps, what tools I like to use, some pro-tips, and resources for you to learn more on your own.

Application Structure

Fundamental background knowledge is important for building any skill, and mobile hacking is no different. Android applications are written primarily in Java, Kotlin (transpiled to Java), and C++. When distributed, they use the .apk extension which stands for Android PackAge. An APK is really just a ZIP file containing all the assets and bytecode for an app. A typical unzipped APK structure looks like this:

```
myapp.apk
├── AndroidManifest.xml
├── META-INF/
├── classes.dex
├── lib/
├── res/
└── resources.arsc
```

Let's briefly cover each of these:

AndroidManifest.xml

This is a compressed version of the AndroidManifest.xml file which contains all of the basic application information such as the package name, package version, externally accessible activities and services, minimum device version, and more. The compressed version of this file is not humanly readable, but there are a couple of tools that are able to uncompress it, most notably being apktool (more on that later).

META-INF/

The META-INF/ folder is essentially a manifest of metadata information including the developer certificate and checksums for all the files contained within an APK. If you were to try and make changes to an APK without removing and re-signing this folder, you would get an error when installing the modified version.

classes.dex

The classes.dex file (sometimes there are multiple) contains all the compiled bytecode of an Android application. Later on, this is what we will decompile into Java source files.

resources.arsc

The resources.arsc file contains metadata about the resources and the XML nodes of the compiled resource files like XML layout files, drawables, strings, and more. It also contains information about their attributes (like width, position, etc) and the resource IDs, which are used globally by both Java and XML app files in the app. This file is compressed into a binary form that is read into memory during runtime. Apktool can also decompress these files and output them into a humanly-readable format for you to explore.

res/

The “res” folder contains compressed binary XML versions of the resource XML files that are paired with the resources.arsc file during runtime to read images, translations, etc. These XML files are in the same binary format as the AndroidManifest.xml file and can be easily decoded with apktool.

lib/

Not all Android apps contain a lib/ folder, but any app with native C++ libraries will. Within this folder, you will find different folders per-architecture, each one containing .so files specifically compiled for that target architecture such as “armeabi-v7a” and “x86”. This is also why you cannot install an app on an x86 device without it providing x86-compiled libs (Google for “INSTALL_FAILED_NO_MATCHING_ABIS”).

Application Analysis

This section is going to focus on the three main methods of analyzing mobile apps — static, passive and dynamic.

Static Analysis

Static analysis is by far the most straightforward way to look at mobile applications, however, it is also the most time-consuming and can take a while to uncover a good bug. One of the most useful things about mobile hacking is that the entire application is distributed when you download it from the Play Store. This means that if you want to know how something in the app works, you just have to find where it happens in the decompiled app code.

On Android, static analysis can be done in a variety of different ways. Personally, there are a few tools that I stick to and use for nearly every single app I am hacking. The first of these tools is [apktool](#).

Two of the most useful features that apktool provides are resource extraction and APK extraction, but many people also like to use it for extracting smali code from a dex file. Smali is just a humanly-readable form of Java bytecode; here's an example of a hello world program in Java and Smali:

Java

```
`import java.lang.System;`

`public class HelloWorld {`
`    public static void main(String[] args) {`
`        System.out.println("Hello World!");`
`    }`
`}`
```

Smali

```
` .class public LHelloWorld;`
`.super Ljava/lang/Object;`
`.source "HelloWorld.java"`

`# direct methods`
`.method public constructor <init>()V`
`    .registers 1`

`    .prologue`
`    .line 3`
`    invoke-direct {p0}, Ljava/lang/Object;-><init>()V`

`    return-void`
`.end method`

`.method public static main([Ljava/lang/String;)V`
`    .registers 3`
`    .param p0, "args"    # [Ljava/lang/String;`
```

```

`    .prologue`
`    .line 5`
`    get-object v0, Ljava/lang/System;-
>out:Ljava/io/PrintStream;`

`    const-string v1, "Hello World!"`

`    invoke-virtual {v0, v1}, Ljava/io/PrintStream;-
>println(Ljava/lang/String;)V`

`    .line 6`
`    return-void`
`.end method`
` `

```

The key advantage of smali is that it is still in the same raw format as the underlying bytecode, but humanly-readable. As such, it's rare to run into misleading smali code since it's usually 1:1 with the dex bytecode. However, I find it to be frustrating to work with, so I try to avoid it unless I come across a major roadblock that forces me to use the raw smali code.

If you know anything about Java reverse engineering, you probably know that it's relatively easy to decompile a .class and .jar file back to source code. However, we do not have a .jar or .class file right now — instead, we have a .dex file, containing raw java bytecode.

Enter [dex2jar](#), a tool that does precisely what the same suggests: converts .dex files into .jar files. You can either manually convert your dex files into jars before decompiling them or use a tool that accepts entire APKs (like jadx). The decompiler that you choose to use from here is often up to your own preference. Personally, I am a fan of [jadx](#), which is an open-source decompiler that has both a GUI and CLI tool, along with some super useful features for decompiling Android apps.

Typically, APKs are minified for easier distribution and obfuscation reasons. Because of this, jadx provides features that make it easier to work with deobfuscated jars and let you enforce minimum field name lengths during decompilation. This means that you can take an app with multiple "a.java" files, and output them as unique class names like "C1234a.java". As a result, you don't have to sort through all the different uses of "a.java", and instead, you can just search for uses of "C1234a.java".

You can instruct jadx do this by using the "--deobf" flag in combination with "--deobf-min".

Example: `./jadx --deobf --deobf-min 3 ...`

Something to realize is that the modified names are based around the original names, so if you see something like io.my.package, the extracted and decompiled io package folder will be changed to something like "p000io" due to "io" being less than three characters. This isn't a super big deal though, and the pros outweigh the cons in my opinion.

Now, I have talked a lot about jadx, but I also mentioned that there are a bunch of other decompilers. I'm not going to cover these individually, but I will list them here and you can explore them on your own time if you'd like:

- [procyon](#)
- [Fernflower](#)
- [CFR](#)

For me, a common decompilation flow looks like this:

1. Use apktool to extract APK and decompress resource files
2. Use jadx to decompile the APK to .java source files
3. Open the decompiled source code folder in Visual Studio Code so I can search and navigate easier

The last thing I will cover with regards to decompilation is a tool called aapt which is built-in to the Android SDK. This tool is quite useful and has the ability to, among other things, dump things like the AndroidManifest.xml tree from an APK without needing to decompile or extract anything. Example:

```
`$ aapt dump xmltree com.myapp-1.0.0.apk
AndroidManifest.xml`
`N: android=http://schemas.android.com/apk/res/android`
`  E: manifest (line=2)`
`    A: android:versionCode(0x0101021b)=(type 0x10)0x409`
`    A: android:versionName(0x0101021c)="1.0.0" (Raw:
"1.0.0")`
`    A: android:installLocation(0x010102b7)=(type
0x10)0x0`
`    A: package="com.myapp" (Raw: "com.myapp")`
`    E: uses-sdk (line=8)`
`      A: android:minSdkVersion(0x0101020c)=(type
0x10)0x15`
`      A: android:targetSdkVersion(0x01010270)=(type
0x10)0x1b`
`    E: uses-feature (line=12)`
`    ...`
```

You can also do this with the aapt dump badging command which can often be simpler:

```
`$ aapt dump badging com.myapp-1.0.0.apk`
`package: name='com.myapp' versionCode='123'
versionName='1.0.0' platformBuildVersionName=''`
`install-location:'auto'`
`sdkVersion:'21'`
`targetSdkVersion:'27'`
`uses-permission: name='com.venmo.permission.C2D_MESSAGE'`
`uses-permission:`
```

```
name='com.google.android.c2dm.permission.RECEIVE'`  
`uses-permission: name='android.permission.INTERNET'`  
`uses-permission:  
name='android.permission.WRITE_EXTERNAL_STORAGE'`  
`uses-permission:  
name='android.permission.READ_EXTERNAL_STORAGE'`  
`uses-permission: name='android.permission.READ_CONTACTS'`  
`...`
```

Passive Analysis

The next type of analysis I am going to cover is called Passive Analysis. Generally speaking, this involves proxying the device, bypassing SSL pinning, and observing device logs.

Logcat

Before getting into proxying and SSL pinning, I'll cover the easier part of this topic first. There is a built-in tool in the Android SDK called logcat which is used to monitor device logs. Often times, you will see apps that print out useful debug information, secret keys, user information, and more, all into logcat. These kinds of things can be very useful when you want to understand what an application is doing. To use logcat, simply run "adb logcat" with a device connected, and you should see system logs. For more info about logcat, check out the docs here:

<https://developer.android.com/studio/command-line/logcat>

Drozer

Next, there is a very popular toolset called [Drozer](#) which was created by MWR InfoSec. Drozer is essentially a toolkit designed to help you analyze Android applications and provides a lot of useful information such as checking for bad permissions, monitoring IPC calls, and more. Personally, I don't use this tool much, but your mileage may vary and I've seen a lot of good things come out of this tool.

SSL Pinning

SSL Certificate pinning is where an app has a known list of valid SSL certificates for a domain (or a set of domains). Then, when making HTTPS connections from the device, it ensures that the certificates from the server match what they are set to in the application. If the cert from the server doesn't match the list of pre-approved certificates, the device drops the connection and throws an SSL error.

There are a lot of different ways to bypass SSL pinning, but the two main ways I bypass it are either with a catch-all Frida script, something pre-built like [JustTrustMe](#) for [Xposed](#), or, if necessary, something totally custom.

First, here is my "catch-all" Frida script — this is not truly universal, but it does a good job in most cases:

<https://gist.github.com/teknogeek/4dc35fb3801bd7f13e5f0da5b784c725>

Usage: frida -U -l universalUnpin.js --no-pause -f com.myapp.name

Proxying

Before bug bounty, I used [Charles Proxy](#), but now I use [Burp Suite Pro](#). Both of these are great choices, and there are tons of other options out there like [Fiddler](#) and [mitmproxy](#). Whatever you choose is totally up to you, but pick what tool you like the most and take advantage of free trials.

Proxying Android Over USB

Proxying over USB is something that I use almost every time I am testing an Android app, whether it be on an emulator or on a physical device. One of the little-known features of adb is that it can tunnel traffic to/from a host machine and a connected device.

This means that you can do things like run a webserver on your laptop, reverse-proxy the port, and access it over a USB connection from your Android device. It also means that if your web proxy is listening on 127.0.0.1:8080 on your laptop, and you don't want to expose the port (or you can't hit it due to NAT), then you can reverse-proxy it to the device directly over USB instead trying to connect by IP.

Here's how to set it up:

1. Make sure your device is connected via ADB
2. adb reverse tcp:8080 tcp:8080
3. Settings → WiFi → Long Press Network → Manage Network → Advanced → Proxy → Manual
4. Proxy Host: 127.0.0.1
5. Proxy Port: 8080
6. Press Save

Now your device should send all traffic to 127.0.0.1:8080 which is then proxied over USB to port 8080 on your host machine. No more spotty connections or wondering if your router is causing blocking proxy connections.

Dynamic Analysis

Dynamic analysis is a way of interacting with and figuring out security vulnerabilities within applications by writing dynamic hooks to talk with them.

I am not going to spend much time talking about the other tools, like Xposed or Substrate, as Frida is far superior at this stage. [Frida](#) is a tool

that allows you to modify, hook, and dynamically interact with applications, hook methods, inspect class variables, and more.

Now...this section could take up a whole blog post on its own, and teaching the fundamentals of Frida is a bit advanced, so I will leave this as an exercise to the reader ;)

By far the most useful resources for writing Frida scripts are:

1. <https://frida.re/docs/android/>
2. <https://www.frida.re/docs/javascript-api/>
3. <https://www.google.com/> (Google is your friend)

If you find yourself writing a Frida script and are stumped further than the internet can help, hit me up on twitter @0xteknogeek and I'll see if I can help (no guarantees).

Do you still have some unanswered questions about Android hacking? Well, you're in luck! We are doing a QA next week and you can still submit your questions using [this form](#).

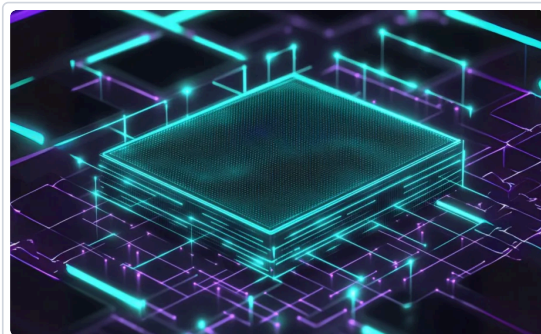
From the HackerOne Blog

Related Blog Posts



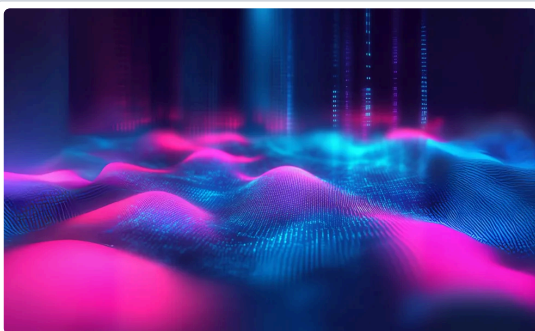
HackerOne News

LLMs adversarial testing with HackerOne: Join the CTF!



Cloud Security

GCP Security Configuration Review and Best Practices



Public Policy

Belgium's NIS2 Transposition: A Practical Model for other EU States

Company

[Leadership](#)

[Careers](#)

[Partners](#)

[Newsroom](#)

[Contact Us](#)

Knowledge Center

[Application Security](#)

[Penetration Testing](#)

[Cloud Security](#)

[Hacking](#)

[Cybersecurity Attacks](#)

[DevSecOps](#)

Resources

[Blog](#)

[Documentation](#)

[Leaderboard](#)

[Partner Portal](#)

[Resources](#)

Contacted by a
hacker? [+](#)

[Cookie Preferences](#)

[Policies](#)

[Terms](#)

[Privacy](#)

[Security](#)

[Trust](#)

©2025 HackerOne All rights reserved.